

ORF335_PSET5-Copy1

April 22, 2026

1 Question 1

calculate implied volatility (ChatGPT helped convert the formulas from the slides into latex and cwrite bs_price_call and bs)price_put)– We will use Newton’s method

$$C_{\text{market}} = C_{\text{BS}}(S, K, r, T, \sigma_{\text{imp}})$$

The vega of a call option is the same as the vega for a put option

$$v = \frac{\partial V}{\partial \sigma} = SN'(d_1)\sqrt{T-t}$$

$$d_1 = \frac{\log\left(\frac{S}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)(T-t)}{\sigma\sqrt{T-t}}$$

```
[330]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
```

```
[331]: calls = pd.read_csv('/Users/leenboualia/Downloads/calls.csv')
puts = pd.read_csv('/Users/leenboualia/Downloads/puts.csv')

calls['mid'] = (calls['bid'] + calls['ask']) / 2
puts['mid'] = (puts['bid'] + puts['ask']) / 2
```

```
[332]: S0 = 6816.89
T = 112 / 365
r = 0.0369
```

```
[333]: def bs_price_call(S, K, r, sigma, T):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
def bs_price_put(S, K, r, sigma, T):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
```

```
return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)
```

```
[102]: def bs_vega(S, K, r, sigma, T):  
    # Vega is the same for calls and puts  
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))  
    return S * norm.pdf(d1) * np.sqrt(T)
```

```
[110]: def implied_vol_newton_call(price, S, K, r, T,  
                                sigma0=0.3, tol=1e-8, max_iter=100):  
    sigma = sigma0  
    for _ in range(max_iter):  
        price_bs = bs_price_call(S, K, r, sigma, T)  
        vega      = bs_vega(S, K, r, sigma, T)  
  
        if abs(vega) < 1e-10:  
            return np.nan  
  
        sigma_new = sigma - (price_bs - price) / vega  
  
        if sigma_new <= 0:  
            sigma = sigma / 2  
            continue  
  
        if abs(sigma_new - sigma) < tol:  
            return sigma_new  
  
        sigma = sigma_new  
        # in case there is no convergence  
    return np.nan  
  
def implied_vol_newton_put(price, S, K, r, T,  
                           sigma0=0.3, tol=1e-8, max_iter=100):  
    sigma = sigma0  
    for _ in range(max_iter):  
        price_bs = bs_price_put(S, K, r, sigma, T)  
        vega      = bs_vega(S, K, r, sigma, T)  
  
        if abs(vega) < 1e-10:  
            return np.nan  
  
        sigma_new = sigma - (price_bs - price) / vega  
  
        if sigma_new <= 0:  
            sigma = sigma / 2  
            continue  
  
        if abs(sigma_new - sigma) < tol:
```

```

        return sigma_new

    sigma = sigma_new
    # in case there is no convergence
    return np.nan

```

```

[111]: calls['iv'] = calls.apply(
        lambda row: implied_vol_newton_call(row['mid'], S0, row['strike'], r, T),
        axis=1)

puts['iv'] = puts.apply(
    lambda row: implied_vol_newton_put(row['mid'], S0, row['strike'], r, T),
    axis=1)

```

```

[112]: # there was a bug because the arrays wouldn't match size, had to ask chat
calls_clean = calls.dropna(subset=['iv'])
puts_clean = puts.dropna(subset=['iv'])

```

```

[334]: fig, ax = plt.subplots(figsize=(11, 6))
ax.scatter(calls_clean['strike'], calls_clean['iv'],
           color='red')
ax.scatter(puts_clean['strike'], puts_clean['iv'],
           color='blue',)

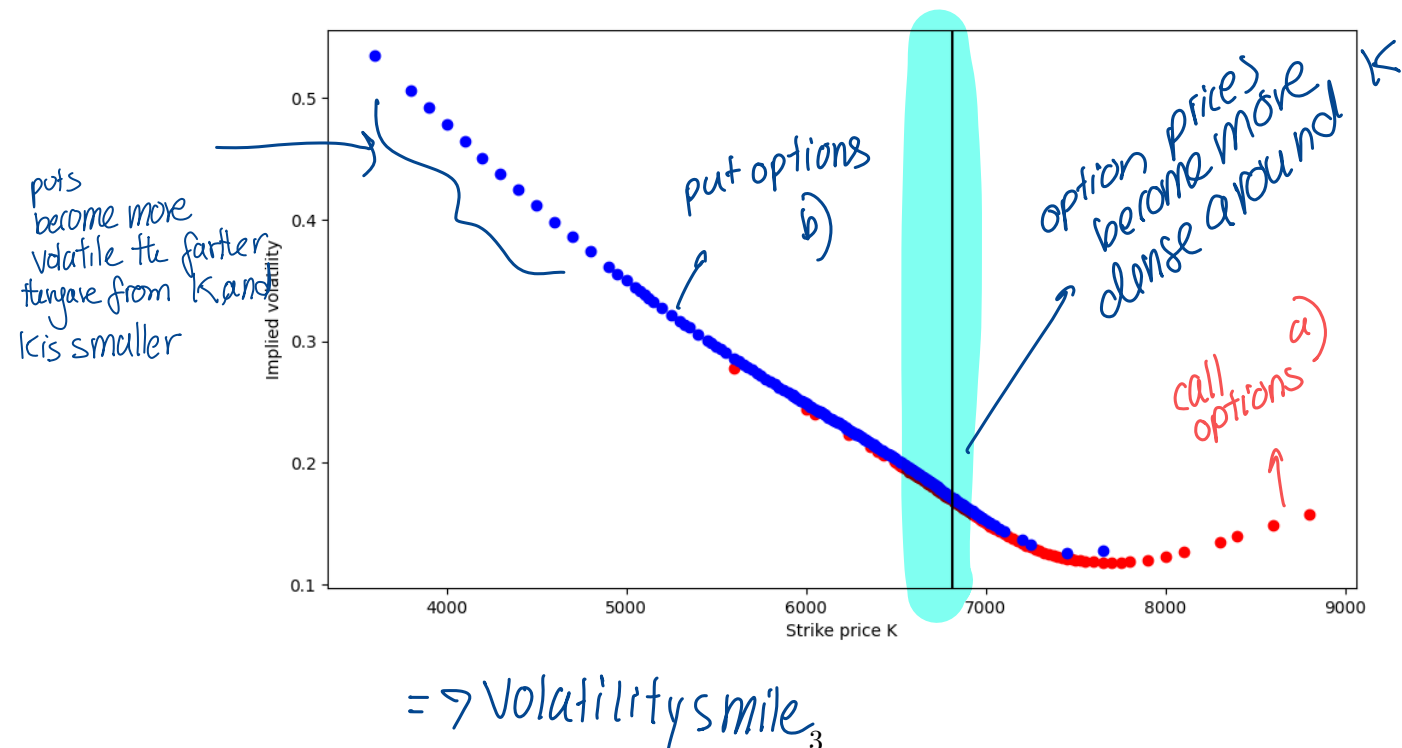
ax.axvline(S0, color='black')
ax.set_xlabel('Strike price K')
ax.set_ylabel('Implied volatility')

```

```

[334]: Text(0, 0.5, 'Implied volatility')

```



Question: Comment on the shape of the resulting graph. In particular, use put-call parity to explain why the implied volatilities from calls and puts should nearly coincide in the region where both are available

Answer: The graph's shape resembles a smile where the put options create the lefthand tail while the call options create the righthand tail. the saddle point of the graph is around 7500. The graph also gets more dense (i.e, the prices of the options become closer to eachother) around the strike price K and there is a form of convergence between the implied volatility of calls and puts since the points become more dense/clustered around when the option is near the money. It also skews to the left hand side meaning that puts have higher volatility when strike prices are lower. The put call parity states that

$$C_0 - P_0 = S_0 - Ke^{-rT}$$

We know that the vega's of the call and put options are the same, so if we take the derivative wrt. sigma for the equation above, it confirms that

$$\frac{dC_0}{d\sigma} - \frac{dP_0}{d\sigma} = \frac{d}{d\sigma} (S_0 - Ke^{-rT}) = 0$$

in this case we are actually using the midpoint price to compute the volatility so the spread between the bid and ask price still satisfies the put call parity because the vegas of the call and put options in the graph are nearly identical when the option is near the money, hence the overlap and clustering around K. At the extreme ends of the curve, there is a big spread between the bid and ask causing the model to deviate and the prices to be drastically different and skew.

2 Problem 2

From lecture slides: Any European option can be replicated by a self-financing portfolio that holds

$$h_t^S = \frac{\partial V}{\partial S}(t, S_t)$$

shares of stock (Delta), and

$$h_t^B = \frac{V(t, S_t) - h_t^S S_t}{B_t}$$

units of the bond. The quantity $\Delta = \frac{\partial V}{\partial S}(t, S_t)$ is called the option's Delta.

```
[154]: # calculate implied volatility
option_price_1= 23.33
S = 603.63
K = 600
r = 0.0369
T= 72/365
implied_vol_2 = implied_vol_newton_call(price=23.33, S=603.63, K=600, r=0.0369,
↪T=72/365)
print("implied volatility 2/12",implied_vol_2)
```

implied volatility 2/12 0.17957487385670923

```
[335]: def bs_delta(S, K,r,T, sigma):  
        d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))  
        return norm.cdf(d1)  
h_s = bs_delta(S=603.63, K=600, r=0.0369, T=72/365, sigma=implied_vol_2)  
print("h_s (black scholes delta)= ",h_s)
```

h_s (black scholes delta)= 0.581905211976562

3 Problem 2 - b)

h_s = 0.581905211976562

h_b = -327.9254431054121

(computed below)

Stock purchase amount (\$) 351.41837656476554

Amount of bond sold (\$) 327.9585966958889

net PnL, i.e value of replicating portoflio 23.459779868876637

```
[165]: h_b = - ((S*h_s) - option_price_1)  
#h_b = -( 0.581905211976562*603.63 -(23.33))/(1)  
print("h_b = ", h_b)  
closing_price_3_12 = 603.91  
print("Stock purchase amount ($)", bs_delta(S=603.63, K=600, r=0.0369, T=72/  
↪365, sigma=0.17957487385670923)*closing_price_3_12)  
print("Amount of bond sold ($)", h_b*-1*(math.e ** (.0369/365)))  
print("net PnL, i.e value of replicating portoflio", h_s *closing_price_3_12 +  
↪h_b*(math.e ** (.0369/365) ))  
portfolio_value = closing_price_3_12*h_s + h_b *math.e ** (r/365)  
print(portfolio_value)
```

h_b = -327.9254431054121

Stock purchase amount (\$) 351.41837656476554

Amount of bond sold (\$) 327.9585966958889

net PnL, i.e value of replicating portoflio 23.459779868876637
23.459779868876637

```
[171]: df = pd.read_csv('/Users/leenboualia/Downloads/hedge.csv')  
spot = df['spot'].tolist()  
option_price = df['option_price'].tolist()
```

```
[173]: N = len(spot)  
implied_vol = np.zeros(len(spot))  
delta_all = np.zeros(len(spot))  
hs_t = np.zeros(len(spot))  
portolfio_value = np.zeros(len(spot))  
hb_t = np.zeros(len(spot))
```

```

T0 = 72

T = T0 / 365
vol0 = implied_vol_newton_call(price=option_price[0], S=spot[0], K=K, r=r, T=T)
d0 = bs_delta(S=spot[0], K=K, r=r, T=T, sigma=vol0)

implied_vol[0] = implied_vol_2
delta_all[0] = d0
hs_t[0] = d0
hb_t[0] = option_price[0] - d0 * spot[0]
portfolio_value[0] = option_price[0]

for i in range(1, N):
    T = (T0 - i) / 365

    dt = 1 / 365
    bond_value = hb_t[i-1] * np.exp(r * dt)
    stock_value = hs_t[i-1] * spot[i]
    portfolio_value[i] = stock_value + bond_value

    vol_i = implied_vol_newton_call(price=option_price[i], S=spot[i], K=K, r=r,
    ↪T=T)
    d_i = bs_delta(S=spot[i], K=K, r=r, T=T, sigma=vol_i)

    implied_vol[i] = vol_i
    delta_all[i] = d_i

    hs_t[i] = d_i
    hb_t[i] = portfolio_value[i] - d_i * spot[i]

```

```

/var/folders/cc/s2_k90vj13n1gv6n8szjgvn40000gn/T/ipykernel_5049/1841054440.py:2:
RuntimeWarning: divide by zero encountered in scalar divide

```

```

    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
/var/folders/cc/s2_k90vj13n1gv6n8szjgvn40000gn/T/ipykernel_5049/2752141735.py:3:
RuntimeWarning: divide by zero encountered in scalar divide

```

```

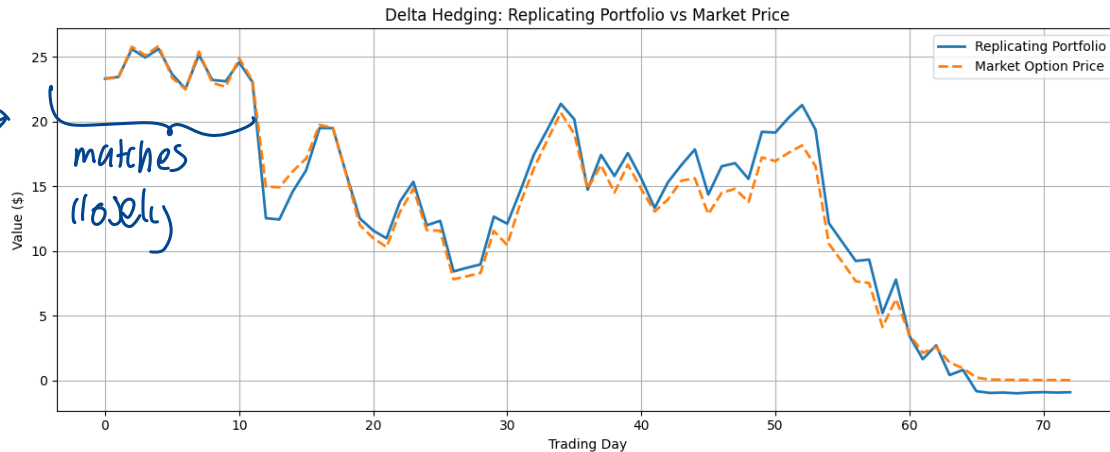
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))

```

```

[174]: plt.figure(figsize=(12, 5))
plt.plot(portfolio_value, label='Replicating Portfolio', linewidth=2)
plt.plot(option_price, label='Market Option Price', linewidth=2, linestyle='--')
plt.xlabel('Trading Day')
plt.ylabel('Value ($)')
plt.title('Delta Hedging: Replicating Portfolio vs Market Price')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



diverges as market option price fluctuates

4 Problem 2 - d)

The replicating portfolio matches the market option price closely in the beginning but then deviates from the market option price.

There could be overnight changes in the option price that cause discontinuous jumps and lead to the discrepancy in calculation AND THE ESTIMATION OF A UNIFORM SPOT PRICE (like in question 1, where we take the average for the bid and ask, here we assume there is a uniform spot price which could cause discrepancies in comparison to the option price)

Since we recalculate delta and volatility everyday, there are shifts in the daily recalibration of the portfolio. Also, the compounding for the interest rate is continuous and assumes that the interest rate is constant but that might not be the case and could contribute to discrepancies

since we assume r is constant but i.r. it could be changing.

5 Problem 3 Markowitz

```
[208]: df = pd.read_csv('/Users/leenboualia/Downloads/techprices.csv')
import cvxpy as cp
```

```
[329]: class StockReturns:
    def __init__(self, file_path):
        df = pd.read_csv(file_path, index_col=0, parse_dates=True)
        returns_df = df.pct_change().dropna()
        for ticker in returns_df.columns:
            setattr(self, ticker, returns_df[ticker].values)

        self.all_returns = returns_df
        avg_per_stock = returns_df.mean() * 252
        cov_matrix = returns_df.cov().values * 252
        # computed using the formula in the lecture notes
```

DAILY SIMPLE RETURNS

mean return vector
{ cov. matrix

```

w= (np.linalg.inv(cov_matrix)@np.ones(67))/(np.ones(67).T@np.linalg.
↳inv(cov_matrix)@np.ones(67))
    print("weights vector for minimum variance ortfolio in # a",w)
    mean_return_portfolio = (avg_per_stock.T@np.linalg.inv(cov_matrix)@np.
↳ones(67))/ ((np.ones(67).T@np.linalg.inv(cov_matrix)@np.ones(67)))
    print("mean return on the minimum variance portfolio_
↳=",mean_return_portfolio)
    variance_portfolio = 1/ ((np.ones(67).T@np.linalg.inv(cov_matrix)@np.
↳ones(67)))
    print("variance on the minimum variance portfolio =",_
↳variance_portfolio)
    print("standard deviation on the minimum variance portfolio =",_
↳variance_portfolio**0.5)
    # optimal mean-variance portfolio
    mu_annual = returns_df.mean().values * 252
    mu = returns_df.mean().values
    gam = 7
    weights_2 = cp.Variable(67)
    objective = cp.Maximize(mu.T @ weights_2 - (gam/2) * cp.
↳quad_form(weights_2, cov_matrix))
    constraints= [cp.sum(weights_2)==1]
    problem = cp.Problem(objective, constraints)
    problem.solve()
    weights_2_values = weights_2.value
    print("optimal weights for minimum variance portoflio with risk_
↳aversion 7 ", weights_2_values)
    # annualized mean return of the optimal protofolio

    annual_return = mu_annual.T @ weights_2_values
    print("mean annual return on optimal minimum variance portoflio",_
↳annual_return)
    # standard deviation of optimal minimum varuanc eportolfio , computed_
↳using formula
    standard_devition_optimal = (weights_2_values.
↳T@cov_matrix@weights_2_values)**0.5
    print("variance on optimal minimum variance portoflio",_
↳standard_devition_optimal**2)
    print("standard deviation on optimal minimum variance portoflio",_
↳standard_devition_optimal)
    # Used chat to impirt the tickers because i dont know how

    tickers = returns_df.columns
    weights_opt = weights_2.value

    plt.figure(figsize=(12,5))
    plt.scatter(tickers, weights_opt, color='red')

```

```

plt.xticks(rotation=90)
plt.title("Optimal Mean-Variance Portfolio Weights ( = 7)")
plt.ylabel("Weight")
plt.tight_layout()
plt.show()
R = 0.01
# compute tangency portfolio , using formula in lecture notes
tangency_portfolio_weights = (np.linalg.inv(cov_matrix) @ (mu_annual -
↪ R*np.ones(67))) / (np.ones(67).T @ np.linalg.inv(cov_matrix) @ (mu_annual - R*np.
↪ ones(67)))
print("tangency portfolio weights = ", tangency_portfolio_weights )
# find tangency portfolio returns and stddev, again using formulae from
↪ lecture notes
mu_tangency = mu_annual.T @ tangency_portfolio_weights
stddev_tangency = np.sqrt( tangency_portfolio_weights.T @ cov_matrix @
↪ tangency_portfolio_weights)
#mu_tangency = (mu_annual.T @ np.linalg.inv(cov_matrix) @ (mu_annual -
↪ R*np.ones(67))) / (np.ones(67).T @ np.linalg.inv(cov_matrix) @ (mu_annual - R*np.
↪ ones(67)))
print("annual return of the tangency portfolio = ", mu_tangency )
# stddev_tangency = ((mu_annual - R*np.ones(67)).T @ np.linalg.
↪ inv(cov_matrix) @ (mu_annual - R*np.ones(67))) / ((np.ones(67).T @ np.linalg.
↪ inv(cov_matrix) @ (mu_annual - R*np.ones(67)))**2)
print(" stddev of the tangency portfolio = ", stddev_tangency )

n = len(weights_opt)

lam = np.linspace(-0.2, 1.5, 20)
weights_frontier = np.zeros((n, 20))
returns_frontier = np.zeros(20)
volatility_frontier = np.zeros(20)
for i in range(20):
    #weights_frontier[:, i] = lam[i] * tangency_portfolio_weights + (1 -
↪ lam[i]) * w
    weights_frontier[:, i] = lam[i] * tangency_portfolio_weights + (1 -
↪ lam[i]) * weights_opt

    returns_frontier[i] = mu_annual.T @ weights_frontier[:, i]

    volatility_frontier[i] = np.sqrt(
        weights_frontier[:, i].T @ cov_matrix @ weights_frontier[:, i]
    )
plt.figure(figsize=(8,5))
plt.plot(volatility_frontier, returns_frontier, marker='o')
plt.plot(0, 0.01, marker='x', color='red')

```

```

plt.text(0, 0.01, "R")

plt.plot(stddev_tangency, mu_tangency, marker='x', color='red')
plt.text(stddev_tangency, mu_tangency, " Tangency Portfolio")
plt.plot(standard_devition_optimal, annual_return, marker='x',
↪color='black')
plt.text(standard_devition_optimal, annual_return, " portfllo with risk_
↪aversion = 7")

plt.plot(variance_portfolio**0.5, mean_return_portfolio, marker='x',
↪color='black')
plt.text(variance_portfolio**0.5, mean_return_portfolio, " portfolio_
↪from a) with minimum variance ")

plt.scatter(np.sqrt(np.diag(cov_matrix)), avg_per_stock , marker='o',
↪color='green')
plt.plot([0, stddev_tangency], [r, mu_tangency])

plt.xlabel("Volatility (Std Dev)")
plt.ylabel("Return")
plt.title("Efficient Frontier")
plt.grid(True)
plt.show()
#ax.scatter(weights_2_values,, color='red')
#ax.set_xlabel('Strike price K')
#ax.set_ylabel('Implied volatility')
market_returns = returns_df.values @ tangency_prtofllo_weights
market_variance = np.var(market_returns)

beta = np.zeros(67)
for i in range(67):
    stock_ret = returns_df.values[:, i]
    beta[i] = np.cov(stock_ret, market_returns, ddof=1)[0, 1] /
↪market_variance

tickers_array = np.array(tickers)
sorted_indices = np.argsort(tickers_array)
sorted_tickers = tickers_array[sorted_indices]
sorted_betas = beta[sorted_indices]

# used to chat to help find argsort and print formatting
print("First 5 ticker-beta pairs:")
for t, b in zip(sorted_tickers[:5], sorted_betas[:5]):
    print(f" {t}:  = {b:.4f}")

print("Last 5 ticker-beta pairs:")
for t, b in zip(sorted_tickers[-5:], sorted_betas[-5:]):
    print(f" {t}:  = {b:.4f}")

```

```

R_daily = R / 252
market_excess = market_returns - R_daily
stocks_excess = returns_df.values - R_daily

market_demeaned = market_excess - market_excess.mean()
stocks_demeaned = stocks_excess - stocks_excess.mean(axis=0)

betas_ols = (market_demeaned @ stocks_demeaned) / (market_demeaned @
↪market_demeaned)
alphas_ols = stocks_excess.mean(axis=0) - betas_ols * market_excess.
↪mean()

sorted_betas_ols = betas_ols[sorted_indices]
sorted_alphas_ols = alphas_ols[sorted_indices]
# used chat's printing format from the question above

print("First 5 OLS estimates = ")
for t, a, b in zip(sorted_tickers[:5], sorted_alphas_ols[:5],
↪sorted_betas_ols[:5]):
    print(f" {t}:    = {a:.6f},    = {b:.4f}")

print("Last 5 OLS estimates = ")
for t, a, b in zip(sorted_tickers[-5:], sorted_alphas_ols[-5:],
↪sorted_betas_ols[-5:]):
    print(f" {t}:    = {a:.6f},    = {b:.4f}")

print("difference between OLS and CAPM betas = ", np.max(np.
↪abs(betas_ols - beta)))
print("The difference is not very significant")

data = StockReturns('/Users/leenboualia/Downloads/techprices.csv')

#print(data.AAPL.mean())
# compute minimum variance portfolio weights

```

```

weights vector for minimum variance ortfolio in # a [ 0.05818943 -0.03899279
-0.02890399  0.04883605 -0.05576592  0.01414522
-0.00175716 -0.01361089 -0.03057312  0.07504118  0.04615013  0.02792703
-0.00991781  0.0873467  -0.02061011  0.1853657  -0.0137773  0.00886502
-0.0468162  -0.03123628  0.02863879 -0.08248934  0.0129328  -0.02931036
 0.10755075 -0.08064373  0.06730737  0.00122791  0.10393266  0.05063752
 0.01415634 -0.06894826  0.0430218  0.17972829 -0.01914838  0.0117687
-0.08276622 -0.04268463  0.0259003  -0.014333  0.10335943 -0.00734326
-0.03968694 -0.01231294 -0.03752434 -0.01049671  0.0131997  0.05881032
 0.04024179  0.03684365  0.04700399  0.12331733 -0.02055947 -0.05595714
 0.02527062 -0.05838853  0.02290142  0.03989573 -0.01192268  0.09609191

```

} a

0.06473878 0.08192797 -0.01690986 0.07756758 0.0071242 -0.02750641
-0.02607037]

mean return on the minimum variance portfolio = 0.10565904538419006

variance on the minimum variance portfolio = 0.01908204985599851

standard deviation on the minimum variance portfolio = 0.13813779300393686

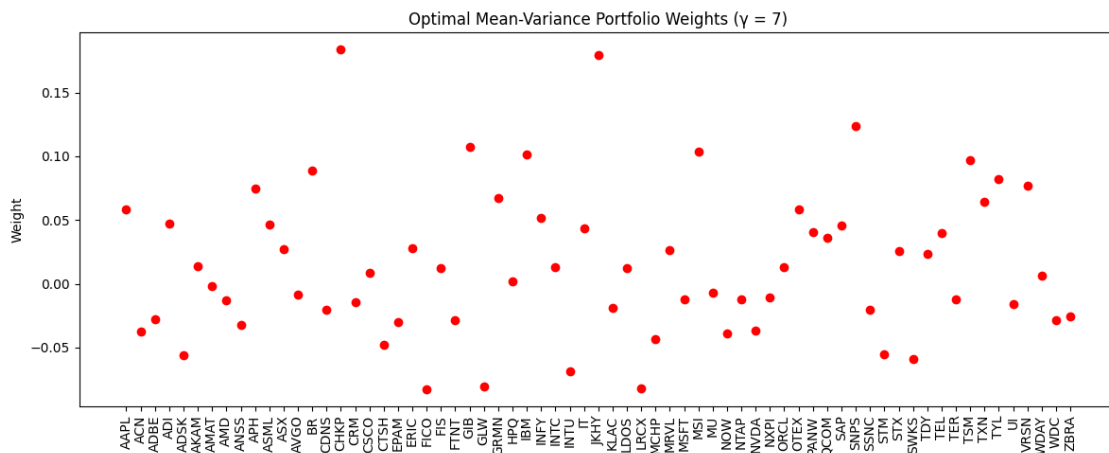
optimal weights for minimum variance portoflio with risk aversion 7 [

0.05838747 -0.03756206 -0.02822087 0.04740188 -0.05590675 0.01358845
-0.00183531 -0.01320831 -0.03208562 0.07496285 0.04631142 0.02714098
-0.00826645 0.08891117 -0.02035285 0.1837106 -0.01455589 0.00865103
-0.04791235 -0.0299398 0.02777988 -0.08273649 0.01258303 -0.02876349
0.10736402 -0.08049736 0.06707349 0.00184646 0.10157645 0.05137653
0.01326683 -0.06860131 0.04311554 0.17961492 -0.0188443 0.01214523
-0.08175166 -0.04313371 0.02624471 -0.01259131 0.10345579 -0.00694526
-0.03906573 -0.01242926 -0.03643145 -0.01043396 0.01298999 0.05857158
0.04045821 0.03584972 0.04536487 0.12343052 -0.02029882 -0.0556832
0.02594083 -0.05875359 0.02358523 0.03994998 -0.01233216 0.09683714
0.06431051 0.08226869 -0.01622677 0.07717672 0.00618773 -0.02866284
-0.02540156]

mean annual return on optimal minimum variance portoflio 0.10949609793450726

variance on optimal minimum variance portoflio 0.019084225055857003

standard deviation on optimal minimum variance portoflio 0.13814566607699644



TAN GENCY PORTFOLIO

tangency portolflio weights = [0.12787586 0.46445315 0.21147277 -0.45582415
-0.1053214 -0.18177321

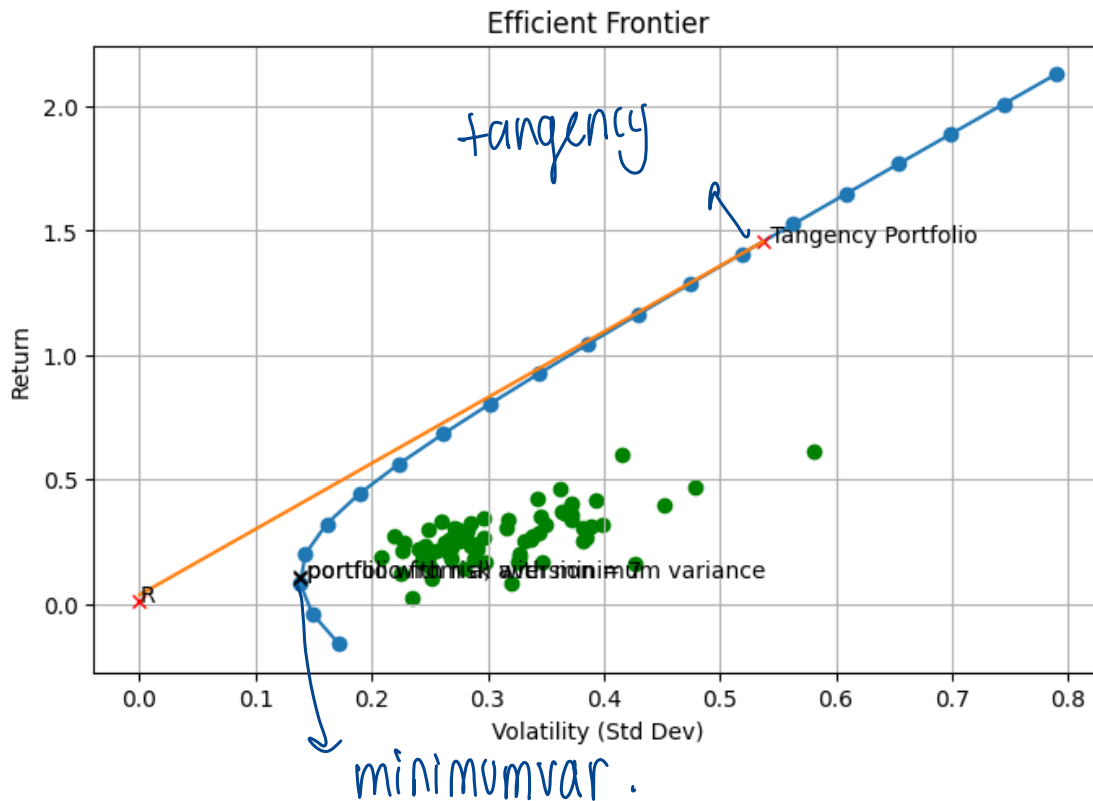
-0.02925686 0.12805252 -0.56279469 0.04747923 0.10290485 -0.24867076
0.57116682 0.63785751 0.06991484 -0.39703519 -0.28775009 -0.06643112
-0.43252998 0.424974 -0.27359615 -0.16945567 -0.11014763 0.16312474
0.04184307 -0.02913914 -0.01499005 0.21888342 -0.72517739 0.31068488
-0.2988484 0.05313852 0.07600912 0.13983497 0.08785304 0.14426206
0.27423811 -0.20070686 0.14709238 0.59853823 0.13726791 0.13270643
0.17890588 -0.053245 0.3470456 0.01158447 -0.06059371 -0.02519802
0.11639518 -0.31290553 -0.52977454 0.16314839 0.07116006 0.04043704

0.26110407 -0.18684498 0.26352196 0.05898646 -0.15601126 0.35832314
 -0.08596175 0.2018195 0.22345663 -0.05996834 -0.32240407 -0.43443376
 0.2092729]

annual return of the tangency portfolio = 1.4558503648600734
 stddev of the tangency portfolio = 0.5370456043302492

(d)

(c)
 and
 (d)



First 5 ticker-beta pairs:

AAPL: = 0.2015
 ACN: = 0.1620
 ADBE: = 0.2308
 ADI: = 0.1452
 ADSK: = 0.1903

Last 5 ticker-beta pairs:

UI: = 0.3191
 VRSN: = 0.1565
 WDAY: = 0.1701
 WDC: = 0.1040
 ZBRA: = 0.2457

First 5 OLS estimates =

AAPL: = 0.000000, = 0.2014
 ACN: = 0.000000, = 0.1619
 ADBE: = 0.000000, = 0.2307
 ADI: = 0.000000, = 0.1451

(e)

(f)

```

ADSK:  = 0.000000,    = 0.1903
Last 5 OLS estimates =
UI:    = 0.000000,    = 0.3189
VRSN:  = 0.000000,    = 0.1564
WDAY:  = 0.000000,    = 0.1700
WDC:   = -0.000000,   = 0.1040
ZBRA:  = 0.000000,    = 0.2456
difference between OLS and CAPM betas = 0.00018454182662314134
The difference is not very significant

```

we know that every other efficient portfolio can be written in the form

$$w = \lambda w_1 + (1 - \lambda) w_2$$

we know that the minimum variance portfolio and the optimal minimum variance portfolio belong to the efficient frontier

we know the capital allocation line has the point R and the tangency portfolio. we know R = (0.01, 0) the y intercept in 0.01 and the tangency portfolio has coordinates (0.5370456043302492, 1.4558503648600734) → the slope of the capital allocation line is = $1.4558503648600734 / (0.5370456043302492 - 0.1) =$

```
[320]: print(1.4558503648600734 / (0.5370456043302492 - 0.1), "slope of tangency_
      ↪ portfolio")
```

3.331117737909965 slope of tangency portfolio

6 Problem 3-d)

capital allocation line : \$ y = 3.331117737909965 x + 0.01\$

[]:

*computation
for frontier*